UD2 · Prueba de Aula · Sintaxis de JavaScript · Versión B

Módulo Desarrollo Web en Entorno Cliente (DEW)

CFGS Desarrollo de Aplicaciones Web, 2.º curso · Curso 2026/27

Nombre y apellidos	
Fecha	
Resultado de aprendizaje	RA2. Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores web.
Criterios de evaluación	b, c, d, e, f, g (y a, h en la ejecución y depuración del código).
Duración	60 minutos (5 de lectura + 55 de trabajo).
Puntuación	10 puntos: Ej. 1 → 2 · Ej. 2 → 2 · Ej. 3 → 3 · Ej. 4 → 3.

Instrucciones

1. Descarga del aula virtual la carpeta prueba-ud2-b y cópiala dentro de tu repositorio dwec-ejercicios, en ut02/prueba-ud2-b. Ábrela en VS Code. Contiene index.html, RESPUESTAS.md, ejercicio2.js, ejercicio3.js, ejercicio4.js y _node-prompt.js.
2. Escribe tu nombre en la cabecera de cada fichero y en RESPUESTAS.md.
3. Puedes ejecutar los programas de dos formas, como en las actividades: con Live Server y la consola (F12), cargando **un solo** <script> cada vez en index.html; o con node ejercicioN.js en la terminal (_node-prompt.js hace que prompt funcione en Node; Intro sin escribir equivale a Aceptar vacío y Ctrl + D a Cancelar).
4. **Material permitido:** los apuntes de la unidad (UD2_SintaxisJS) y tus propias actividades de ut02. **No permitido:** asistentes de IA, mensajería ni compartir ficheros. El código que entregues debe ser tuyo.
5. Normas de estilo (se aplican a todos los ejercicios): 'use strict' al principio, solo const y let, nombres descriptivos en camelCase, todas las entradas de prompt convertidas con Number() donde corresponda. Pasa npx eslint . antes de entregar.
6. **Entrega:** antes de que acabe el tiempo, git add, git commit -m "Prueba UD2 versión B" y git push. Sube también la carpeta comprimida en la tarea del aula virtual. Un programa que no ejecuta por un error de sintaxis se corrige, pero no puede alcanzar la puntuación máxima del ejercicio.

Organiza el tiempo: unos 10 minutos para el ejercicio 1, 10 para el 2, 15 para el 3 y 20 para el 4. Si te atascas en uno, pasa al siguiente.

Ejercicio 1 · Predice la salida (2 puntos · CE b, c, d)

Para cada fragmento escribe en RESPUESTAS.md qué muestra la consola, línea a línea, o qué error se produce (indica el tipo: ReferenceError, TypeError…), y una justificación de una línea con el concepto de la unidad que lo explica (hoisting, ámbito de bloque, sombreado, valores *falsy*, ??…). Cada fragmento se ejecuta por separado.

Puedes comprobarlo ejecutándolo, pero la mitad de la puntuación está en la justificación.

F1

```
for (var i = 0; i < 3; i++) {}
console.log(i);
for (let j = 0; j < 3; j++) {}
console.log(j);
```

F2

```
let x = 1;
{
  console.log(x);
  let x = 2;
}
```

F3

```
const nivel = 'alto';
{
  const nivel = 'bajo';
}
console.log(nivel);
nivel = 'medio';
```

F4

```
console.log('3' * '4');
console.log('3' + 4 + 5);
console.log(3 + 4 + '5');
```

F5

```
const nombre = '';
console.log(nombre || 'anónimo');
console.log(nombre ?? 'anónimo');
console.log(null ?? 'anónimo');
```

Ejercicio 2 · Completa el código (2 puntos · CE b, d, g)

El fichero ejercicio2.js desglosa un importe en billetes y monedas. Le faltan ocho fragmentos, marcados con los comentarios ① a ⑧. Sustituye cada hueco por el código que falta sin cambiar nada más. El programa debe:

a) Pedir un importe entero en euros. Si el usuario cancela, se usa '100' por defecto (operador ??).

b) Convertir el texto a número.

c) Si el importe no es un número positivo, mostrar un error con console.error y no calcular nada (cláusula de guarda).

d) Calcular cuántos billetes de 50, de 20 y de 10 hacen falta y cuánto queda en monedas (Math.trunc para los billetes completos y % para lo que sobra).

e) Mostrar el desglose y el importe con IGIC del 7 % con dos decimales (toFixed; la constante TIPO_IGIC = 0.07 ya está declarada).

Fragmento del fichero (los huecos están marcados):

```
const importeTexto = prompt('Importe en euros (entero):') /* ① */ ; // '100' si cancela
const importe = /* ② */ ; // texto -> número

if ( /* ③ */ ) {
  console.error('Error: el importe debe ser un número positivo.');
} else {
  const billetes50 = /* ④ */ ; // billetes de 50 completos
  const restoTras50 = /* ⑤ */ ; // lo que sobra tras los de 50
  const billetes20 = /* ⑥ */ ; // billetes de 20 completos
  const restoTras20 = restoTras50 % 20;
  const billetes10 = Math.trunc(restoTras20 / 10);
  const monedas = /* ⑦ */ ; // lo que sobra tras los de 10
  const conIgic = /* ⑧ */ ; // importe + 7 %, dos decimales (texto)

  console.log(`Importe: ${importe} € (con IGIC: ${conIgic} €)`);
  console.log(`Billetes de 50: ${billetes50} · de 20: ${billetes20} · de 10: ${billetes10}`);
  console.log(`En monedas: ${monedas} €`);
}
```

Comprobación: 185 → 3 de 50, 1 de 20, 1 de 10, 5 € en monedas, 197.95 con IGIC · Cancelar → 100 → 2 de 50, 0, 0, 0 €, 107.00 · hola o -5 → mensaje de error.

Ejercicio 3 · Escribe una función (3 puntos · CE d, e, g)

En ejercicio3.js, escribe la función costeAparcamiento(minutos, vehiculo) que calcula lo que cuesta una estancia en un aparcamiento, y un pequeño programa que la use.

Reglas de la función

- Validación (cláusula de guarda): minutos debe ser un número mayor o igual que 0 y menor o igual que 1440 (un día); vehiculo debe ser 'coche', 'moto' o 'furgoneta', en cualquier combinación de mayúsculas y con espacios alrededor (normalízalo con trim() y toLowerCase()). Si algo no es válido, la función devuelve null y no muestra nada por consola.
- Tarifa base por tiempo (if…else if): hasta 15 minutos → gratis (0 €); hasta 60 minutos → 1,50 €; hasta 180 minutos → 1,50 € más 0,04 € por cada minuto por encima de 60; más de 180 minutos → 7,00 € (tarifa máxima).
- Multiplicador por vehículo (switch): coche × 1 · moto × 0,5 · furgoneta × 1,5.
- Devuelve el coste como número (sin redondear). Documenta la función con JSDoc (@param, @returns).

Programa: pide los minutos y el vehículo con prompt, llama a la función y muestra el resultado con un ternario: Datos no válidos o Coste del aparcamiento: X € con dos decimales. Si el usuario cancela, el resultado es Datos no válidos.

Comprueba estos casos y anota la columna «obtenido» en el comentario final del fichero:

minutos	vehiculo	Esperado
10	coche	0,00
60	' Moto '	0,75
90	coche	2,70
200	furgoneta	10,50
1500	coche	null
30	bici	null

Ejercicio 4 · Amplía el código (3 puntos · CE e, f, g)

ejercicio4.js es una mini consola con menú (do…while + switch) que ya tiene dos opciones funcionando y una función de apoyo pedirNumero(mensaje, min, max), que devuelve un número válido o null si el usuario cancela. No modifiques pedirNumero ni las opciones 1 y 2. Añade la opción 3, «Secuencia de Collatz»:

a) Escribe la función contarPasosCollatz(numero), que recibe un entero positivo y devuelve cuántos pasos hacen falta para llegar a 1 aplicando esta regla en un bucle while: si el número es par se divide entre 2; si es impar se multiplica por 3 y se suma 1 (patrón contador). Documéntala. Ejemplo: contarPasosCollatz(6) → 8 (6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1). Si el número ya es 1, devuelve 0.

b) Escribe la función opcionCollatz(): pide un entero positivo con pedirNumero (si devuelve null, muestra «Operación cancelada.» y vuelve al menú), calcula los pasos con contarPasosCollatz y muestra en una sola línea cuántos pasos son y de qué tipo es la secuencia (if…else if): corta (menos de 10 pasos), media (de 10 a 49) o larga (50 o más). Ejemplo: 6 llega a 1 en 8 pasos (secuencia corta).

c) Conecta la opción al menú: añade la línea 3. Secuencia de Collatz al texto del menú y el case 3 correspondiente en el switch, sin romper el funcionamiento de las demás opciones ni de «0. Salir».

Comprobación: 1 → 0 pasos, corta · 6 → 8 pasos, corta · 7 → 16 pasos, media · 27 → 111 pasos, larga · Cancelar → «Operación cancelada.» y vuelve el menú.

Antes de entregar: ¿has puesto tu nombre en los cuatro ficheros? ¿Ejecutan los tres programas sin errores? ¿Has hecho commit y push?